

Lab 7: Drone IMU Motion Monitor

ECE 3301 – Microcontrollers | Fall 2026

Instructor: Sasoun Torousian

1. Introduction

In Lab 5, you developed an interrupt-driven Drone Mission Timing and Safety Supervisor. Your PIC18F46K22 could respond to important events while continuing to execute the main program. You used GPIO, interrupts, timers, UART communication, and a state machine to control the drone through the SAFE, ARMING, ARMED, and FAULT states. However, a real drone cannot make decisions using buttons and timers alone. It must also monitor its physical movement. For example, before a drone takes off, the flight controller must determine whether the aircraft is stationary, tilted, or experiencing unusual movement. During operation, the controller must continuously monitor sensor information to identify potentially unsafe conditions. In this lab, you will add a motion sensor to your existing drone safety controller using the SPI communication protocol introduced in Chapter 8. You will configure the PIC18F46K22 to communicate with an LIS3DH three-axis accelerometer, read acceleration measurements along the X, Y, and Z axes, and use those measurements to determine the motion condition of your drone.

Important: Difference Between the Lecture and This Lab

Chapter 8 introduced SPI communication using the ICM-20948 IMU, which contains a three-axis accelerometer, a three-axis gyroscope, and a magnetometer. However, this laboratory uses the LIS3DH, which contains only a three-axis accelerometer. Both devices support SPI communication, but they have different register maps, configuration procedures, and measurement capabilities. Therefore, you cannot directly copy the ICM-20948 code from Chapter 8 and expect it to work with the LIS3DH. You are required to read the LIS3DH datasheet, identify the necessary registers, understand how the device communicates through SPI, and develop your own LIS3DH driver using the SPI concepts learned in Chapter 8. The lecture provides the foundation, but you are responsible for adapting that knowledge to a different device. The LIS3DH does not contain a gyroscope. Therefore, this lab will focus on measuring acceleration, detecting tilt, identifying movement, and recognizing excessive acceleration. You will not measure angular velocity or implement gyroscope-based motion detection.

2. Objectives

After completing this lab, you should be able to configure the PIC18F46K22 MSSP peripheral for SPI communication, interpret the LIS3DH datasheet, identify the registers needed to initialize and operate the accelerometer, perform SPI register reads and writes, verify communication using the WHO_AM_I register, retrieve signed acceleration measurements from all three axes, and use those measurements to classify the motion condition of your drone. You will also integrate the accelerometer into the interrupt-driven architecture developed in Lab 5, schedule sensor measurements using Timer2, transmit acceleration information through UART, and connect the motion-monitoring subsystem to your existing drone safety state machine. The main objective is to develop a functioning motion-monitoring subsystem without replacing the safety controller you have already built.

3. Required Hardware

Component	Purpose
PIC18F46K22 Curiosity HPC Board	Main drone controller
Adafruit LIS3DH STEMMA QT breakout (PID 2809)	Three-axis accelerometer
CP2102 USB-to-TTL adapter	UART communication with the computer
ARM and FAULT pushbuttons	Drone safety inputs
LEDs and current-limiting resistors	Drone status indicators
Breadboard and jumper wires	Circuit connections
USB cable	Programming and communication

The LIS3DH is a low-voltage sensor. For this lab, use the approved Adafruit LIS3DH breakout board rather than a bare LIS3DH IC. The breakout includes supporting circuitry that makes it easier to interface with a microcontroller. You must distinguish between the voltage requirements of the LIS3DH IC and those of the breakout board. Verify the breakout's power-input and logic-level specifications before connecting it to the Curiosity HPC board. For the specified Adafruit breakout, use its documented VIN input and onboard level-shifted SPI connections. Do not assume that a different LIS3DH module provides the same protection.

4. What You Are Building

You will extend your existing Lab 5 safety controller by adding a motion-monitoring subsystem. The LIS3DH will measure acceleration along three axes and send the measurements to the PIC18F46K22 through SPI. Your program will process these measurements and determine whether the drone is stable, moving, tilted, or experiencing excessive acceleration. The accelerometer will not replace the existing safety inputs. Instead, it will provide additional information that your drone controller can use when making decisions. Your completed system must preserve the SAFE, ARMING, ARMED, and FAULT operating states. It must also introduce a separate motion state so that the controller can independently track the operating condition of the drone and the movement detected by the accelerometer.

5. LIS3DH SPI Wiring

The PIC18F46K22 contains an MSSP peripheral that supports SPI communication. You will use the same SPI signals introduced in Chapter 8, but you must connect them to the corresponding pins on the LIS3DH breakout board.

PIC18F46K22	SPI Function	LIS3DH Breakout
RC3	SCK	SCL/SCK
RC5	SDO (MOSI)	SDA/SDI
RC4	SDI (MISO)	SDO
RD0	GPIO output	CS
GND	Ground	GND
Approved power supply	Power	VIN

The SCK signal provides the clock used to synchronize communication between the PIC and the LIS3DH. The PIC transmits commands and data through SDO, while the LIS3DH returns information through the PIC's SDI input. The RD0 pin will control Chip Select (CS), which determines when the LIS3DH is selected for SPI communication. Before starting any SPI transaction, CS must be HIGH. To communicate with the sensor, the PIC must drive CS LOW, perform the required SPI transfers, and then return CS HIGH. Keep your SPI wires short and verify that the sensor and microcontroller share a common ground. Long wires and poor connections can introduce noise and cause unreliable communication. Use the breakout's SPI-compatible connections as documented by its manufacturer. In particular, confirm that the breakout is placed in SPI mode and that its SDI and SDO pins are correctly identified.

6. Preserve Your Lab 5 Safety Controller

Start with your working Lab 5 program. Do not create an entirely new application that ignores the safety features developed in previous laboratories. Your program must continue to use the following operating states:

```
typedef enum
{
    SAFE,
    ARMING,
    ARMED,
    FAULT
} DroneState;
```

The emergency FAULT input must continue to have the highest priority. The ARM condition must remain valid for two seconds before the controller enters the ARMED state, and the communication watchdog must continue to operate as implemented in Lab 5. Your Timer2 interrupt must also continue to provide the system time base. Remember that an interrupt service routine should execute quickly because other important events may need attention. Do not perform SPI transactions inside the Timer2 interrupt. Instead, use the interrupt to schedule accelerometer measurements and allow the main program to perform the actual SPI communication. For example, your existing Timer2 interrupt can set a flag every 10 milliseconds:

```
volatile uint8_t accel_sample_flag = 0;
volatile uint32_t system_ms = 0;

/* Inside the existing Timer2 interrupt handler */

if (PIR1bits.TMR2IF)
{
    PIR1bits.TMR2IF = 0;

    system_ms++;

    if ((system_ms % 10) == 0)
    {
        accel_sample_flag = 1;
    }
}
```

This example assumes that Timer2 has already been configured to generate a one-millisecond system tick. You must preserve the other interrupt-handling functions from Lab 5 rather than replacing the entire ISR with this example.

7. Part 1 – Initialize SPI Communication

Begin by configuring the PIC18F46K22 MSSP peripheral for SPI communication. Use the SPI initialization concepts introduced in Chapter 8, including the TRIS registers, SSP1CON1, SSP1STAT, and the SPI clock configuration. Your program should contain functions similar to:

```
void SPI_Init(void);

uint8_t SPI_Transfer(uint8_t data);
```

The `SPI_Init()` function must configure the appropriate pin directions, enable the MSSP peripheral, and select an SPI clock frequency supported by both the PIC18F46K22 and the LIS3DH. The `SPI_Transfer()` function must transmit one byte through SSP1BUF, wait for the transfer to complete, and return the received byte. Remember that SPI is a full-duplex communication protocol. Every time the PIC transmits one byte, it simultaneously receives one byte. This is why the PIC must transmit a dummy byte when it wants to read data from the LIS3DH. Before writing your SPI initialization code, read the LIS3DH datasheet and determine which SPI mode is required. You must identify the correct clock polarity and clock phase and configure the PIC accordingly.

8. Part 2 – Read the LIS3DH Datasheet

Unlike Chapter 8, where the ICM-20948 register configuration was explained during the lecture, this lab requires you to identify the LIS3DH registers independently. Download the LIS3DH datasheet from STMicroelectronics and study the sections covering the SPI interface, register map, control registers, output data registers, and measurement resolution. Before writing the sensor driver, identify the following information.

Register or Feature	What You Must Determine
WHO_AM_I	Register address and expected device identification value
CTRL_REG1	How to enable X, Y, and Z axes and select the output data rate
CTRL_REG4	How to select full-scale range and measurement resolution
OUT_X_L / OUT_X_H	How to read X-axis acceleration
OUT_Y_L / OUT_Y_H	How to read Y-axis acceleration
OUT_Z_L / OUT_Z_H	How to read Z-axis acceleration
SPI command format	How to distinguish read and write operations
Multiple-byte access	How to read consecutive registers in one SPI transaction

The LIS3DH uses a different register organization from the ICM-20948. In particular, you do not need to implement the ICM-20948 register-bank selection procedure. You must also investigate the LIS3DH's measurement modes and determine how the selected mode affects the format and resolution of the acceleration data. Do not begin the motion-classification portion of the lab until you understand how to configure the sensor and retrieve valid acceleration measurements.

9. Part 3 – Verify SPI Communication

Your first programming objective is to prove that the PIC18F46K22 can communicate with the LIS3DH. The LIS3DH contains a WHO_AM_I register that returns a fixed identification value. Reading this register allows your program to determine whether the expected device is connected and responding. Create a function similar to:

```
uint8_t LIS3DH_ReadRegister(uint8_t reg);
```

Your function must control CS, transmit the correct SPI read command, receive the register value, and then release CS. For the LIS3DH, the WHO_AM_I register address is `0x0F`, and the expected identification value is `0x33`. You should independently verify these values in the datasheet. Your program must compare the returned value with the expected value and transmit an appropriate UART message. For example:

```
LIS3DH DETECTED
WHO_AM_I = 0x33
```

If the returned value is incorrect, display an error message and prevent the system from entering the ARMED state. Do not assume that SPI communication is working simply because your program compiles. You must demonstrate that the PIC receives the correct identification value from the physical sensor.

10. Part 4 – Configure the LIS3DH Accelerometer

After verifying SPI communication, initialize the LIS3DH so that it can continuously measure acceleration. Create the following functions:

```
void LIS3DH_WriteRegister(uint8_t reg, uint8_t data);

uint8_t LIS3DH_Init(void);
```

Your initialization function must configure the accelerometer to enable all three measurement axes, select an appropriate output data rate, and choose a full-scale acceleration range. For this lab, use the following target configuration:

Parameter	Required Setting
Measurement axes	X, Y, Z enabled
Output data rate	100 Hz
Full-scale range	±2 g
Measurement mode	High-resolution mode
Communication	Four-wire SPI
Block data update	Enabled

You must determine the correct register values from the LIS3DH datasheet. Do not copy the ICM-20948 configuration values because the two sensors use different register definitions. After writing the configuration registers, read them back to verify that the LIS3DH accepted your settings. Your initialization function should return a success or failure indication so that the main program can respond appropriately if sensor configuration fails.

11. Part 5 – Read Three-Axis Acceleration

The LIS3DH measures acceleration along the X, Y, and Z axes. These measurements represent the acceleration experienced by the sensor, including the effect of gravity. Create variables to store the three acceleration measurements:

```
int16_t ax;
int16_t ay;
int16_t az;
```

Each axis uses two output registers containing the low and high portions of the measurement. Unlike the ICM-20948 example from Chapter 8, the LIS3DH stores the low byte first and the high byte second. For a two-byte measurement, you can combine the values as follows:

```
uint16_t raw;

raw = ((uint16_t)high_byte << 8) | low_byte;
```

You must then interpret the combined data according to the LIS3DH measurement mode. In the required high-resolution mode, the valid signed acceleration measurement is left-justified in the 16-bit register pair, so you must account for the unused lower bits and sign extension before converting the measurement into acceleration units. Create a function similar to:

```
void LIS3DH_ReadAccel(
    int16_t *ax,
    int16_t *ay,
    int16_t *az
);
```

Use a multiple-byte SPI read to retrieve the six acceleration data bytes efficiently. The LIS3DH requires the appropriate auto-increment command bit when reading consecutive registers through SPI. You must determine how to set this bit using the datasheet. When the sensor is stationary on a horizontal surface, the magnitude of the acceleration vector should be approximately 1 g because the accelerometer measures the effect of gravity. Rotate the sensor and observe how the acceleration values change along the three axes. You should notice that the axis aligned with gravity produces the largest measurement.

12. Part 6 – Convert Raw Measurements into Acceleration

Raw acceleration values are useful for debugging, but converting them into physical units makes the measurements easier to understand. The LIS3DH supports different full-scale ranges, including ±2 g, ±4 g, ±8 g, and ±16 g. The sensitivity depends on the selected range and measurement mode. For the required ±2 g high-resolution configuration, the nominal sensitivity is 1 mg per count when the 12-bit measurement has been correctly aligned and sign-extended. You can calculate acceleration in units of g using:

$a_x = frac$

14. What You Must Demonstrate

During the lab demonstration, you must show the instructor that the complete system works.

Step	Your Action	Expected Result
1	Power ON the system.	PIC initializes and reports `SAFE`.
2	Read LIS3DH `WHO_AM_I`.	UART displays `0x33` and confirms successful sensor detection.
3	Place the sensor flat on a table.	Motion state displays `STABLE`, total acceleration approximately 1 g.
4	Move the sensor.	Motion state changes to `MOVING`.
5	Tilt the sensor approximately 45°.	Motion state changes to `TILTED`.

Step	Your Action	Expected Result
6	Press ARM and complete the two-second arming sequence.	Drone state changes to `ARMED`.
7	Move the sensor normally.	Drone remains `ARMED`.
8	Generate brief excessive acceleration.	Drone remains `ARMED`; no immediate fault.
9	Generate excessive acceleration continuously for at least 200 ms.	Drone enters `FAULT`.
10	Reset to SAFE, then press the emergency FAULT button.	Drone immediately enters `FAULT`.
11	Show UART telemetry.	Serial terminal displays drone state, motion state, and X/Y/Z acceleration at approximately 10 Hz.

15. What You Must Submit

Submit your complete MPLAB X/XC8 source code, including the SPI driver, LIS3DH register configuration, acceleration processing, motion classification, and integration with your existing safety controller. Your code must include comments explaining the important register settings and experimentally selected motion thresholds. Also submit a screenshot of your UART terminal showing successful LIS3DH detection and at least three different motion conditions. No formal lab report is required.

16. Important Expectations

You are not required to build a complete drone flight controller or implement PID control. You are also not required to measure angular velocity because the LIS3DH does not contain a gyroscope. However, simply displaying acceleration values is not sufficient. Your program must interpret the measurements, classify motion, and use that information to make safety decisions. The most challenging part of this lab is developing the LIS3DH driver from the datasheet rather than copying the ICM-20948 code from Chapter 8. One important limitation: The LIS3DH measures acceleration, not angular velocity. You cannot reliably detect every kind of motion using this sensor. For this lab, MOVING means detectable changes in acceleration, not proof of all physical movement. Likewise, sustained excessive acceleration should be demonstrated using controlled, repeatable movement rather than arbitrary shaking. The minimum acceptable demonstration is a working SPI interface, valid three-axis acceleration measurements, four motion classifications, functioning UART telemetry, and successful integration with the Lab 5 safety state machine.